PROJECT SUBMISSION REPORT

CloudLens: Cloud-Native Image Gallery

Full-Stack Enterprise Web Application with AWS S3, EC2, Node.js, React & PostgreSQL

Candidate Name:	**Samriddha Das**
Project Topic:	Cloud-Native Image Gallery using AWS S3, EC2, Node.js, React & PostgreSQL
Live Web Application URL:	**http://32.236.11.79**
GitHub Repository:	**https://github.com/samriddha-coder/-CloudLens-AWS-hyphen-gallery-**
Compute Infrastructure:	Amazon Web Services (AWS) EC2 Ubuntu 24.04 LTS (Instance: ap-southeast-2)
Object Storage Bucket:	Amazon S3 (Bucket: samriddhabaket1718 | Region: ap-southeast-2)
Database Service:	PostgreSQL 16 Engine with persistent volume storage (Dockerized)
DevOps Orchestration:	Multi-Container Docker & Docker Compose Cluster

1. Executive Summary

CloudLens is a scalable, cloud-native image repository and management web application built to address modern enterprise requirements for high-durability media storage, role-based security, and microservices architecture. The system enables public visitors to upload and explore media assets while restricting destructive administrative tasks (image deletion) strictly to authenticated administrator accounts. The solution leverages Amazon Web Services (AWS S3 and EC2) alongside Docker orchestration and PostgreSQL to guarantee zero single-point-of-failure and rapid scalability.

2. System Architecture & Component Design

The project decouples presentation, application business logic, relational metadata, and binary object storage into modular tiers:

Component	Technology	Architectural Responsibility
Frontend UI	React.js (Vite), CSS3, Lucide Icons	Single Page Application (SPA), drag-and-drop uploader, real-time preview, lightbox modal & admin authentication.
Backend API	Node.js, Express.js REST API	Multipart stream processing via Multer memory buffers, JWT verification, and PostgreSQL queries.
Object Storage	Amazon S3 (ap-southeast-2)	Direct streaming of image assets using @aws-sdk/client-s3 with custom bucket and CORS policies.
Database	PostgreSQL 16 Engine	Relational schema tracking image IDs, original filenames, S3 URLs, uploader identities, and timestamps.
Compute / Server	AWS EC2 (Ubuntu Linux)	Virtual host in ap-southeast-2 executing all microservice containers with public IPv4 routing.

Containerization	Docker & Docker Compose	Automated building, isolated networking, volume persistence, and service health monitoring.

3. Core Functional Capabilities

3.1 Public User Capabilities (Upload & View):

- **Drag-and-Drop Image Ingestion:** Supports high-resolution JPG, PNG, WEBP, GIF, and SVG formats up to 10MB.
- **Direct S3 In-Memory Streaming:** Files are read directly into server memory buffers and streamed to S3 without writing intermediate files to local disk, saving compute IOPS.
- **Responsive Gallery Grid:** High-speed interactive grid showing image title, uploader name, format, and timestamp.
- **Full-Screen Lightbox Preview:** Modal view for high-definition asset inspection with direct links to original S3 objects.

3.2 Administrator Security & RBAC:

- **Restricted Deletion:** Public users observe locked indicator badges. Destructive deletion requires an authorized JWT.
- **Admin Modal Authentication:** Secured login using Bcrypt password hashing and token expiry.
- **Synchronized Two-Phase Deletion:** Deleting an asset removes the file from the AWS S3 bucket and prunes metadata from PostgreSQL.

4. AWS Infrastructure & Deployment Configuration

4.1 Amazon S3 Bucket Configuration:

- **Bucket Name:** samriddhabaket1718 (Region: ap-southeast-2)
- **Bucket Policy:** Configured with public read permissions (s3:GetObject) allowing global low-latency browser rendering.
- **CORS Configuration:** Enabled GET, PUT, POST, DELETE, and HEAD verbs for web browser origins.
- **IAM Role & Access:** Provisioned programmatic IAM credentials bound with AmazonS3FullAccess policy.

4.2 Amazon EC2 Networking & Security Groups:

- **Virtual Machine:** Ubuntu 24.04 LTS instance with allocated Public IPv4 **32.236.11.79**.
- **Inbound Port Rules:** Port 80 (HTTP Production Web), Port 3000 (React Frontend Web), Port 5000 (Express REST API), Port 22 (SSH Management).

5. Docker Container Orchestration

All application services run as isolated containers within a unified Docker Compose network:

1. **gallery_postgres:** PostgreSQL 16 Alpine container with host volume mounting for database durability and automated healthchecks.

2. **gallery_backend:** Node.js runtime connected to the internal Docker PostgreSQL network and the external AWS S3 API.
3. **gallery_frontend:** Multi-stage Nginx Alpine container serving optimized React production builds with reverse-proxy routing.

6. Verification Results & Conclusion

The application has been thoroughly tested and verified. Live testing confirms that files uploaded from the web browser stream seamlessly to AWS S3, metadata persists in PostgreSQL across container lifecycles, and administrator credentials successfully execute atomic asset deletions. The solution represents a robust, cloud-native software architecture adhering to industry best practices.